Contents

1. Sorting and Searching

1.1. Concepts

1.1.1. Bit Operations

In `c++`, you can perform binary operations on individual bits. This may sound confusing, so let's look at some examples.

1.1.1.1. Negative Numbers

In a computer, all numbers are stored in binary. For an `int`, the computer allocated 32 bits. The number 5 stored in an `int` actually looks like:

$$00000000000000000000000000000101$$

Now for an `unsigned int`, the largest number you can store would be $2^{32} - 1 = 4,294,967,295$, which in binary would be:

$$11111111111111111111111111111111$$

However regular `int` need the capability to store negative numbers. If we start from 0 and then subtract 1, we roll back and reach -1 which would be:

$$11111111111111111111111111111111$$

Going back one more place give's -2:

$$11111111111111111111111111111110$$

And so on. until $-2^{31} = -2147483648$:

$$10000000000000000000000000000000$$

if you subtract one from this you go to $2^{31} - 1 = 2147483647$

$$01111111111111111111111111111111$$

The way to convert from binary to decimal using this signed format is the realised the rightmost bit represents -2^{31} instead of positive 2^{31}. So to write a negative number in binary, you can set the rightmost bit to 1 and then find what number to add to -2^{31} to get $-n$. This would be $2^{31} - n$. Finding this is a pain however, so there's a much better way:

Say we want to find out what is -9 in binary. First we must take the **1′s complement** of positive 9. This simply means we flip every bit (0′s become 1′s and 1′s become 0′s):

$$\begin{aligned} 9 &= 00000000000000000000000000001001 \\ &11111111111111111111111111110110 = -10 \end{aligned}$$

If you positive number was n, this will give you $-n - 1$. To get $-n$, you must add 1. This is called taking the **2′s complement**. This would give you:

$$-9 = 11111111111111111111111111110111$$

1.1.1.2. AND `(&)`

Let's say I have the numbers 5 and 7. The question is what would be the output to `cout << (5 & 7);`?

First we write 5 and 6 in binary, which become 0101 and 0110. Then we perform the `and` operation on each bit to get a new number in binary:

$$\begin{array}{r} 0101 \\ \texttt{\&}\ 0110 \\ \hline 0100 \end{array}$$

Finally convert 0100 back to decimal, which is 4

So the code `cout << (5 & 6);` would output `4`

1.1.1.3. OR (|)

Now we want to find out the output of `cout << (5 | 6) << endl`. We now perform the `or` operation on each bit

$$\begin{array}{r} 0101 \\ \texttt{|}\ 0110 \\ \hline 0111 \end{array}$$

Which is 3 in decimal.

1.1.1.4. XOR (^)

Now we want to find out the output of `cout << (5 ^ 6) << endl`. We now perform the `xor` operation on each bit

$$\begin{array}{r} 0101 \\ \texttt{\^{}}\ 0110 \\ \hline 0011 \end{array}$$

Which is 7 in decimal.

1.1.1.5. NOT(~)

The `not(~)` operator flips all the bits of a number. In the earlier examples, we we're only showing 4 bits because the numbers were small. However, the `int` type has a total of 32 bits. So if you want to find the output of `cout << (~5) << endl` The answer would be:

$$\begin{array}{r} \sim 00000000000000000000000000000101 \\ \hline 11111111111111111111111111111010 \end{array}$$

Which is ~~`4294967290`~~ `-6`. This is because ~ generates the $1'$s complement which for some positive n will give you $-n-1$.

1.1.1.6. Left shift(<<) and right shift(>>)

Left shifting is moving all the bits some number of places to the left. Each left shift is just multiplying the number by 2. So `cout << (3 << 4);` would be $000011 \rightarrow 000110 \rightarrow 001100 \rightarrow 011000 \rightarrow 110000$ which is $3 \times 2^4 = 3 \times 16 = 48$. Right shifting works in the exact opposite manner. Each right shift gives you the floor of the number divided by 2 ($\lfloor \frac{n}{2} \rfloor$). So `cout << (57 >> 3);` is $111001 \rightarrow 011001 \rightarrow 001100 \rightarrow 000110 = 6$.

1.1.1.7. Lowest Set Bit (LSB)

The lowest set bit is the value of the rightmost bit of a binary number that is set to 1. This bit contributes the least to the number. For example, the number 20 in binary is 10100. The right most bit is in the second position from the left (0-indexed). The value it represents is $100 = 2^2 = 4$ which means the LSB(20) = 4.

If you want to find the LSB(n), all you have to do is `n & -n`.

Why does this work? For that, you must first break up `-n` into `~(n+1)` because that's taking the 2′s complement. When you take the 1′s complement of n (`~n`) the rightmost 1 becomes the rightmost 0. All bit to the right of this 0 are 1′s.

If you now add 1 to get the 2′s complement. All the ones up to the rightmost 0 become 0′s and the rightmost 0 become a 1. So now when you take the `and` of `n` and `-n`, the bits just before the rightmost one have all been flipped, so `&` will make them all 0′s. Only the rightmost bit is 1 in both `n` and `-n` so it will be preserved. This will give you a new number in binary which is the LSB(n).

Here's how it looks on 20:

$$\begin{aligned} 20 &= 00000000000000000000000000010100 \\ \& -20 &= 11111111111111111111111111101100 \\ \hline & 00000000000000000000000000000100 \end{aligned}$$

1.1.2. Bitmask

Bitmasking is the technique of using the binary representation of numbers to represent subsets of the question. Let's look at a problem which can be solved using bitmasks.

Say your given an array and want to return all possible subsets of that array. Here's the code on how to do that and then we'll go through the code:

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int main(){

  int n = 3;
  vector<int> v = {5, 4, 7};

  for(int mask = 0; mask < (1 << n); mask++){//mask takes all values from 0 - 2^n-1.

    cout << "{ ";

    for(int i = 0; i < n; i++){
      if(mask & (1 << i)//checking if the ith of the mask is 1
        cout << v[i] << " ";
    }

    cout << "}" << endl;
  }

  return 0;
}
```

In the code, the variable `mask` goes through all subsets, where each subset is numbered from 0 to $2^n - 1$. In this case $n = 3$ so `mask` goes from 0 to 7. Then for each value of mask, you output all the elements `v[i]` where the `i`th bit(from right to left) is true. This will generate the following output.

```
1 { }
2 { 5 }
3 { 4 }
4 { 5 4 }
5 { 7 }
6 { 5 7 }
7 { 4 7 }
8 { 5 4 7 }
```

1.1.3. Prefix sum

Let's say your asked this question. You're given an array of numbers, and then your given some queries. Each query will give you a range. Your goal is to output the sum of all numbers in that range. For example, let's say you have the following array:

$$\{5, -6, 4, 3, 12, 6, -7, -3\}$$

And now you're told to find the sum of elements from index index 4-7, index 2-5, index 1-3. The answers to that would be:

$$12 + 6 + -7 + -3 = 8$$
$$4 + 3 + 12 + 6 = 25$$
$$-6 + 4 + 3 = 1$$

Note that indices are 0-indexed.

You could solve this questions by simply iterating through all elements in each range and then adding them up. However, each of these operations is amortized $O(n)$. If there are q queries, your complexity would be $O(nq)$. If n and q's limits are 2×10^5, $O(nq)$ would be too slow.

The must faster way would be to compute a **prefix sum** array. This means that every element `pref[i]` sore the sum of all elements from `v[0]` to `v[i]`. Now let's say you want to know the sum from index `a` to `b`. You only have to compute `pref[b] - pref[a-1]` to get the answer. Using our example, the prefix sum array would be:

$$\text{original array } \{5, -6, 4, 3, 12, 6, -7, -3\}$$
$$\text{prefix sum array } \{5, -1, 3, 6, 18, 24, 17, 14\}$$

Now the answers to the 3 queries are:

$$14 - 6 = 8$$
$$24 - (-1) = 25$$
$$6 - 5 = 1$$

You get the correct answer by only having to subtract 2 numbers rather than having to add an entire array.

Here's the code for the implementation of prefix sum:

```cpp
#include <bits/stdc++.h>
using namespace std;

int main(){

  int n, q;
  cin >> n >> q;

  vector<int> v(n);
  for(int i = 0; i < n; i++)
    cin >> v[i];

  vector<int> pref(n);
  pref[0] = v[0];

  for(int i = 1; i < n; i++)
    pref[i] = v[i] + pref[i-1];

  for(int i = 0; i < q; i++){
    int a, b;
    cin >> a >> b;

    if(a != 0)
      cout << (pref[b] - pref[a-1]) << endl;
  }

  return 0;
}
```

Sample input:

```
8 3
5 -6 4 3 12 6 -7 -3
4 7
2 5
1 3
```

Output:

```
8
25
1
```

The space complexity is $O(n)$ and both update and query operations run in $O(\log n)$ time.

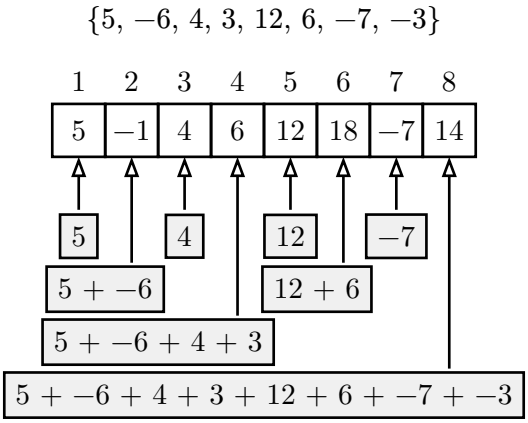
1.1.4. Binary Indexed Tree

Let's say for the question in the previous section, we not only want the ability to find the sum in a given range, we also want to update an element in the array. This means that we need to be able to both change values in the array and output the sum in any given range quickly.

Our earlier approach of maintaining a prefix sum fails, because even though we can output the sum of elements in a range in $O(1)$. If we even change a single value, the time it takes to generate the whole array is amortized $O(n)$. For the constraints of $n \leq 2 * 10^5, q \leq 2 * 10^5$, this is too slow.

There is a data structure which can help us do updates and sums in $O(n \log(n))$. This is called a **binary indexed tree(BIT)** or a **Fenwick tree**. In a Fenwick tree, each element is 1-indexed. The ith value in the Fenwick tree stores the sum of all elements in the original array from $i - \text{LSSB(i)} + 1$ to i. (see Section 1.1.1.7 for meaning of LSSB).

To understand this better, let's look at the array from our previous example and the Fenwick tree that's made from the array.



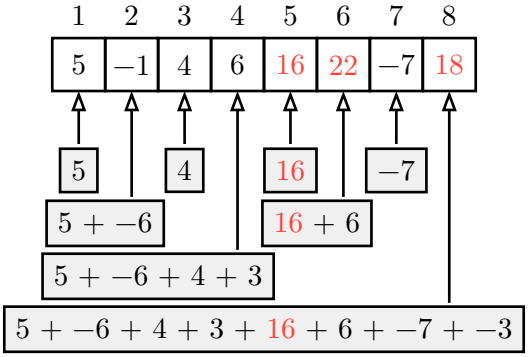
Note that the values in a Fenwick tree are one-indexed, so there we be an empty element at `fenw[0]`.

Hopefully the image made it clearer on how data is stored. The reason for storing data like this is because if you want to add a value to 1 element in the original array, you only need to update $O(\log n)$ values in the Fenwick tree. And after doing this, you can find the sum in $O(\log n)$. Say we wish to add 4 to the 5th index (one-indexed), we only need to update the 5th, 6th, 8th

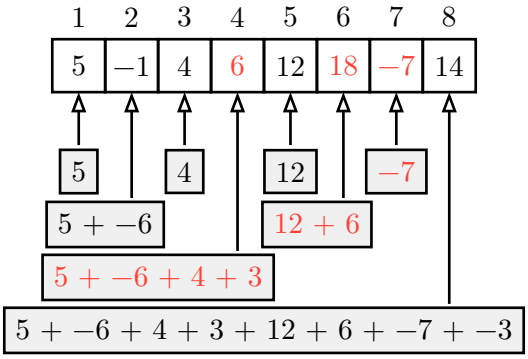
index. If you now want to compute the prefix sum of the array from index 7, you only need to add the values in the 7th, 6th, 4th index.

Here's a diagram to illustrate the changes:

Add 4 to the 5th index:



Prefix sum at the 7th index:



If you can calculate the prefix sum at some index i in $O(\log n)$, you can then do `sum(b) - sum(a-1)` to find the sum of numbers in the subarray `a` to `b`.

Here's the code for the Fenwick tree implementation:

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int n;
vector<int> fenw;

void add(int x, int k){
  for(; x <= n ; x += x & -x)// x & -x is the LSSB(x)
    fenw[x] += k;
}

int sum(int x){
  int ans = 0;
  for(; x > 0; x -= x & -x)// x & -x is the LSSB(x)
    ans += fenw[x];
  return ans;
}

int sum(int a, int b){
  return sum(b) - sum(a - 1);
}
```

```
22
23 int main(){
24   int q;
25   cin >> n >> q;
26   fenw.resize(n + 1, 0);
27   for(int i = 1; i <= n; i++){
28     int x;
29     cin >> x;
30     add(i, x);
31   }
32
33   for(int i = 0; i < q; i++){
34     int t;
35     cin >> t;
36     if(t == 1){// addition queries
37       int x, k;
38       cin >> x >> k;
39       add(x, k);
40     }
41     else if(t == 2){// range sum queries
42       int a, b;
43       cin >> a >> b;
44       int ans = sum(a, b);
45       cout << sum(a, b) << endl;
46     }
47   }
48   return 0;
49 }
```

Sample input:

```
8 6
5 -6 4 3 12 6 -7 -3
2 4 7
1 5 4
2 4 7
2 1 3
1 3 -8
2 2 7
```

Output:

```
14
18
3
8
```

1.1.4.1. Fenwick Tree's as Indexed Sets

A Fenwick tree can also be used as an indexed set. In Section 1.1.8, a set was explained to be a data structure that lets you insert, find, and erase elements in $O(\log n)$ time. It was also sorted and contained unique elements. However, it's not possible to simply access the 2nd, or 5th, or 12th, value in a set unless you iterate all the way from the beginning to that position. That makes accessing elements at a specific index $O(n)$.

If you also want to be able to access elements at specific indexes in a set, you can use a Fenwick tree as a frequency table. This means that at `fewn[x]`, you store the of times element `x` occurs in your original data. Of course `fewn[x]` will actually store the sum of frequencies from `x - LSSB(x) + 1` to `x` but you get the point. This makes is sorted by default because it describes how many times 1 appears, followed by how many times 2 appears and so on.

Now if you want to add an element `a` to the set, you simply do `add(a,1)` in the Fenwick tree to increase its frequency by 1. If you want to remove an element `b`, you do `add(b, -1)` to decrease its frequency by 1. If you want to find the index of the last occurrence of an element `c`, do `sum(c)`, if you want to find the index of the first occurrence of `c` do `sum(c-1) + 1`. And finally, the main difference is the ability to find what element is at position `i`. This requires a new function.

Here's the code of the search function:

```C++
int search(int idx){
  int ans = 0;

  for(int k = floor(log2(n)); k >= 0; k--){//go through the powers of 2.
    if(1 << k <= n && fenw[ans + (1 << k)] < idx){//this element is before the idx.
      ans += 1 << k;//update the answer.
      idx -= fenw[ans];//account for all indices upto fenw[ans].
    }
  }

  return ans + 1;//ans was the value that was before idx, so one value ahead of that is at idx.
}
```

In the code of the search function, you start with `k = floor(log2(n))` where 2^k is the largest power of 2 less than or equal to n. Then for each value, you check to see if it's index (`fenw[ans + 1 << k]`) is less than the `idx`. If it is, you add 2^k to the answer and then subtract `idx` by the indexes covered (`fenw[ans]`).

Since `ans` store the number that is definitely before index, `ans + 1` tells you what number is exactly at `idx`. The reason why you can't find the index directly is because the Fenwick tree frequency table can have multiple of the same numbers, so you can't guarantee that you will find what value is there at your exact `idx`.

Finally there is one more thing required to make a Fenwick tree useful as an indexed set. A normal sets size is based on the amount of input n which makes its space complexity $O(n)$.

However, a Fenwick tree is build on the frequency table of the data, which makes it have a space complexity of $O(a)$ where a is the largest input. Usually $n \leq 2 \times 10^5$ however a can be as large as 10^9! This would require around **30 gigabytes** of data, which is way past the memory limits of a question. The solution to this problem is do **index compression**. Because the amount of data is going to be small, we simply remap all the large numbers down to smaller numbers.

For example, say you have the following array:

$$\{3, 10, 4, 5, 2, 2\}$$

If we were to store this array as an indexed set, it would require the storage of $10 + 1$(because 1 indexed) `ints` of storage. Notice how that are are only 5 unique numbers in this entire vector $(2, 3, 4, 5, 10)$. If we were to resign these numbers to just $(1, 2, 3, 4, 5)$, our indexed set would only take $5 + 1$(because 1 indexed) `ints` of memory. This technique is called **index compression**.

1.1.4.2. Index compression

To perform index compression, you need to sort the original vector of values stored in a different vector. Let's call this other vector `comp`. Then remove all the duplicate elements from `comp`. Then to compress the indices, find at what index values from the original vector appear in `comp`. This can be done efficiently with `lower_bound()` because `comp` is sorted. For the above example, comp would look like:

0	1	2	3	4
2	3	4	5	10

Because of `comp`, 2 will get mapped to $0 + 1 = 1$(Because 1 indexing for the Fenwick tree), 3 gets mapped to $1 + 1 = 2$ and so on.

Here's the code for the implementation of index compression:

```cpp
#include <bits/stdc++.h>
using namespace std;

int main(){
  int n;
  cin >> n;
  vector<int> v;
  for(int i = 0; i < n; i++)
    cin >> v[i];

  vector<int> comp = v;
  sort(comp.begin(), comp.end());
  comp.erase(unique(comp.begin(),comp.end()), comp.end());
  for(int i = 0; i < n; i++)
    v[i] = lower_bound(comp.begin(), comp.end(), v[i]) - comp.begin() + 1;//
    lower_bound - comp.begin() gives you the index and +1 makes sure it's one
    indexed.
```

```
16    return 0;
17 }
```

Sample Input:

```
3 10 4 5 2 2
```

Output:

```
2 5 2 3 1 1
```

The code uses the `std::unique()` function, which in a sorted vector, moves all duplicate elements to the end and returns a pointer at the start of the duplicate elements. When then use this pointer and erase all the duplicate elements till the end to generate `comp` correctly.

To compress all the values in `v`, we get an iterator to their position in `comp` using `lower_bound()` and then subtract it with `comp.begin()` to get it's position 0-indexed. We then add 1 to get the compressed value such that it is 1-indexed.

Here's a code will fully summarizes the process of an indexed set:

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int n;
vector<int> fenw;

void add(int x, int k){
  for(; x <= n ; x += x & -x)// x & -x is the LSSB(x)
    fenw[x] += k;
}

int sum(int x){
  int ans = 0;
  for(; x > 0; x -= x & -x)// x & -x is the LSSB(x)
    ans += fenw[x];
  return ans;
}

int sum(int a, int b){
  return sum(b) - sum(a - 1);
}

int search(int idx){
  int ans = 0;

```

```
26    for(int k = floor(log2(n)); k >= 0; k--){//go through the powers of 2.
27      if(1 << k <= n && fenw[ans + (1 << k)] < idx){//this element is before
        the idx.
28        ans += 1 << k;//update the answer.
29        idx -= fenw[ans];//account for all indices upto fenw[ans].
30      }
31    }
32
33    return ans + 1;//ans was the value that was before idx, so one value ahead
      of that is at idx.
34
35  int main(){
36    int n;
37    cin >> n;
38    vector<int> v;
39    for(int i = 0; i < n; i++)
40      cin >> v[i];
41
42    vector<int> comp = v;
43    sort(comp.begin(), comp.end());
44    comp.erase(unique(comp.begin(),comp.end()), comp.end());
45    for(int i = 0; i < n; i++)
46      v[i] = lower_bound(comp.begin(), comp.end(), v[i]) - comp.begin() + 1;
47
48    for(int i = 0; i < n; i++)
49      add(v[i], 1);
50
51    //Now the fen vector is fully ready to behave as an indexed set.
52
53    return 0;
54  }
```

1.1.5. Linked List

A linked list is a data structure, where ever element in a list list has a value and a pointer to the next element. This makes removing elements at a given position $O(1)$ because you only have to make the element before the erased one, point to the element after the erased one. The same is true for inserting an element in a given position.

Linked list can either be made linear or circular. In a linear linked list, the last element points to the `nullptr`. In a circular linked list, the last element points back to the first element.

Here's the implementation of a linear linked list:

```
1                                                                    C++
2   struct Node{
```

```
3     int val;// the value in the current element
4     Node* nxt;//a pointer to the next element
5     Node(const int& val = 0, Node* nxt = nullptr){//constructor
6       this->val = val;
7       this->nxt = nxt;
8     }
9   };
10
11  struct List{
12    Node* head;
13
14    List(const int& size = 0, const int& val = 0){//creaing the linked list.
      The list will have "size" elements filled with val.
15      head = new Node();
16      Node* cur = head;
17      for(int i = 1; i <= size; i++){
18        cur->val = val;
19        cur = cur->nxt = new Node();
20      }
21    }
22
23    void insert(Node* pos, const int& val){//inserts a new node after poss
24    Node* nxt = pos->nxt;
25      pos->nxt = new Node(val,nxt);
26    }
27
28    void erase(Node* pos){//Erases the next node after pos
29      if(pos->nxt == nullptr)
30        return;
31      Node* nxt = pos->nxt;
32      pos->nxt = nxt->nxt;
33      delete(nxt);
34    }
35  }
```

Of course this Is a very poor implementation with not much memory safety leading to memory leaks. Fortunately c++ has it's own implementation of a linked list.

1.1.5.1. `std::list`

Here's a cod examples on how the c++ implementation of a linked list is used:

```C++
1   #include <bits/stdc++.h>
2   using namespace std;
3
4   int main(){
```

```
5     int n;
6     cin >> n;
7
8     list<int> l(n, 0);
9     for(list<int>::iterator it = l.begin(); it != l.end(); it++){
10      int x;
11      cin >> x;
12      *it = x;
13    }
14
15    for(list<int>::iterator it = l.begin(); it != l.end(); it++)// loop to
      erase every odd element(1-indexed).
16      it = l.erase(it);
17
18    for(list<int>::iterator it = l.begin(); it != l.end(); it++)
19      l.insert(it, *it-1);//before each element insert the value of that
        element-1.
20
21    for(list<int>::iterator it = l.begin(); it != l.end(); it++)
22      cout << *it << " ";
23
24    return 0;
25  }
```

Sample input:

```
6
4 -6 3 8 7 -2
```

Output:

```
-5 -6 9 8 -1 -2
```

As you can see from the code, if you want to store a value you simple update `*it`. If you want to insert a value before the current iterator, do `l.insert(it, val)`. Lastly if you want to erase the current iterator, do `l.erase(it)`. `erase()` also return the position to the next iterator so that you don't invalidate your current iterator.

For the `std::list` documentation, click here.

1.1.6. Queue

A **queue** behaves very similarly to a queue in real life. Say you wish to buy tickets for a movie. You must first join the back of the queue, then the people who joined before you must all receive their tickets and then you can buy your own ticket and then leave the front of the queue.

In c++, joining the queue is called **pushing** an element into the queue. Leaving the front of the queue is called being **popped** from the queue.

The data structure of a **queue** has already been implemented in `c++` as `std::queue`.

Some of the operations a `queue` is:

1. `push()` adds an element to the back of the queue in $O(1)$ time.
2. `pop()`: removes the element from the front of the queue in $O(1)$ time.
3. `front()` gets the value of the element at the front without removing it in $O(1)$ time.

Let's look at a practical problem that demonstrates how queues work:

Problem: You are managing a ticket counter. People arrive and join the queue. Every person has a name and the number of tickets they want. Process each person in the order they arrived, and print their information when serving them.

Solution:

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Person{
  string name;
  int tickets;

  Person();// default constructor

  Person(string name, int tickets){
    this->name = name;
    this->tickets = tickets;
  }
};

int main(){
  int n;
  cin >> n;

  queue<Person> q;

  // Adding people to the queue
  for(int i = 0; i < n; i++){
    string name;
    int tickets;
    cin >> name >> tickets;

    q.push(Person(name, tickets));// Add person to the back of the queue
  }

  cout << "Serving customers:" << endl;

  // Process the queue
```

```
34    while(!q.empty()){// While the queue is not empty
35      Person cur = q.front();// Get the person at the front
36      q.pop();// Remove them from the queue
37
38      cout << "Serving " << cur.name << " " << cur.tickets;
39      if(cur.tickets = 1)
40        cout << " ticket." << endl;
41      else
42        cout << " tickets." << endl;
43    }
44
45    return 0;
46  }
```

Sample input:

```
5
Alice 2
Bob 1
Charlie 3
Diana 2
Eve 1
```

Output:

```
Serving customers:
Serving Alice 2 tickets.
Serving Bob 1 ticket.
Serving Charlie 3 tickets.
Serving Diana 2 tickets.
Serving Eve - 1 ticket.
```

As you can see, the people are served in exactly the same order they joined the queue. Alice joined first, so she was served first, and Eve joined last, so she was served last.

While this example could've been achieved with a `vector`, you'll find that there are better uses for queue in the graph algorithm section.

For the `std::queue` documentation, click here.

1.1.7. Greedy algorithms

A greedy algorithm is a type of algorithm where the solution for a smaller subpart of the question also applies to the whole question. A greedy algorithm never goes back and corrects it's previous decision. Let's take a look at a question that can be solve with a greedy algorithm:

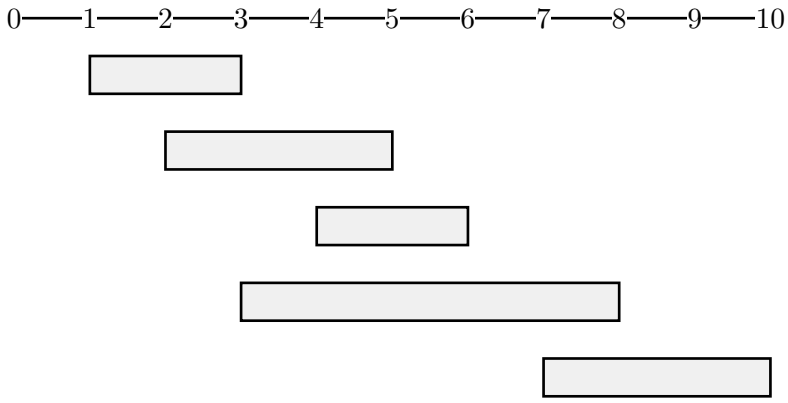
Question: You are given a list of events. Every event has a start time and an end time. You can only attend one event at a time. Your goal is to pick events in such a way so that you can attend the maximum number of events.

The algorithm to solve this question is to pick an event which ends the earliest and then pick the next non overlapping event that ends the earliest and so on. This always ensures that you can pick the largest number of events.

The reason for this is to think of the opposite. If you were to pick an event the ends later, at best case you can still pick the same number of new events. However at worst you will overlap some events that you could have picked. Let's look at the following example:

start	end
1	3
2	5
4	6
3	8
7	10

Here's the visualization of all the events:



These events are currently sorted in ascending order of their end times. Let's say instead of following the strategy by picking the first event $\{1, 3\}$, you were to pick the second event $\{2, 5\}$ which has a later start time. The only new event you can also attend is $\{7, 10\}$. If you were to pick $\{1, 3\}$, you can pick $\{7, 10\}$, but can also pick $\{4, 6\}$. This is why it's always better to pick event's which end earlier because you have nothing to loose and everything to gain.

There are many other questions where you can use a greedy approach and you'll understand how to use them by solving such questions.

(Add tag to Tasks and deadlines when documents are merged)

1.1.8. Sets

A `set` in a data structure in `c++`, which has the following properties:

1. A new element can be added to a `set` in $O(\log n)$ time.
2. An element can be found in $O(\log n)$ time.
3. An element can be removed in $O(\log n)$ time.
4. All elements are sorted in ascending order.
5. All elements in a `set` are unique

Here's a quick problem, whose solution will explain how to use sets:-

Accept numbers from a user. Then check if a number exists in the list, if it does, print `YES` followed by removing that number from the list, otherwise print `NO`. At the end print the new list in ascending order.

Solution:

```C++
#include <bits/stdc++.h>
using namespace std;

int main(){

  int n, q;
  cin >> n >> q;
  set<int> s;

  for(int i = 0; i < n; i++){
    int x;
    cin >> x;
    s.insert(x);//Inserts a value into the set.
  }

  for(int i = 0; i < q; i++){
    int x;
    cin >> x;
    if(s.find(x) != s.end()){//s.find(x) returns the position of x in the set.
      cout << "YES" << endl;
      s.erase(x);//s.erase(x) removes x from the set
    }
    else
      cout << "NO" << endl;
  }

  for(set<int>::iterator it = s.begin(); it != s.end(); it++){
    cout << *it << " ";
  }
  return 0;
}
```

In the solution, we can see that

- `s.insert(x)` inserts `x` into the `set s`. This will ensure that `s` will remain sorted by inserting it into the correct place.
- `s.find(x)` returns the position of `x` in `s`. If `x` doesn't exist, it will return `s.end()` which is a pointer at one place past the position of the last element in the set.
- `s.erase(x)` removes `x` from `s`.

Finally we end up printing all values that are currently in `s`. However, you may notice that instead of the traditional loop with a variable `i` that increase, we're using a

`set<int>::iterator`. An iterator is simply a pointer that is used the go over a data structure that is not traditionally indexed. You can very much use the same syntax with vectors too, but it's not necessary.

1.1.8.1. `lower_bound` and `upper_bound`

Unlike for vectors, if you try to use the `lower_bound()` and `upper_bound()` functions, it won't execute binary search and will instead search through them in linear time. The reason for this is that set iterators are not random access, i.e. you can't just say `it + 5` and get the element 5 places ahead of `it`. Instead, you must run a loop to do `it++` 5 times. Fortunately, `set's` have their own implementation of `lower_bound()` and `upper_bound()`. If you have a `set<int> s`, then s.lower_bound(t) will return an iterator to the lower bound of `t` and `s.upper_bound(t)` will return an iterator to the upper bound of `t`.

1.1.8.2. `multiset`

A `multiset` is exactly like a set except that it can store multiple of the same elements, whereas a `set` does not store duplicates. The syntax for using a `multiset` is identical to a `set`, just write `multiset` instead of set

1.1.8.3. `unordered_set`

An `unordered_set` works a bit different then a set. It supports the following operations

1. A new element can be added to a `unordered_set` in $O(1)$* time.
2. An element can be found in $O(1)$* time.
3. An element can be removed in $O(1)$* time.
4. The order of elements are random.
5. All elements in a `unordered_set` are unique

Notice how it almost identical to a set other than the fact that it faster with the downside of no sorted order. This looks as if it would be useful to use an `unordered_set` instead of a `set` if you just want to check if elements exists or not due to their $O(1)$ vs the much slower $O(\log n)$. However, this $O(1)$ is not guaranteed and for large test cases that you may expect during questions, it usually ends being the much worse $O(n)$ which will lead to a Time Limit Exceeded(TLE). This is why you should always use a `set` over an `unordered_set` even if you don't care about the sorting order.

1.1.8.4. `unordered_multiset`

Again, it's the same as an `unordered_set` except that it can store multiple of the same element. This also has $O(1)$ operations with the caveat that its worse case is $O(n)$. So you should use `multiset` over `unordered_multiset`.

1.1.9. Lambda expressions

Lambda expressions are a way to write functions in line without having to write them separately. For example:

```C++
#include <bits/stdc++.h>
using namespace std;

int main(){

```

```
6     int n;
7     cin >> n;
8
9     function<int (int)> fact = [&] (int num){ // defining the lambda expression
10      if(num == 1)
11        return 1;
12      return num * fact(num-1);
13    };
14
15    cout << fact(n) << endl;
16
17    return 0;
18  }
```

As you can see we've defined a function within the main function. The first part `function<int (int)>` says that you're making a function with return type int and one int parameter. Then after the equal to the `[&]` part allows you to access variables in the scope of the outer function by reference. `[=]` would allow you to access them by value and `[]` wouldn't allow any access. Then you write the actual contents of the function inside the braces.

Lambda expressions are also useful to just make temporary functions without having to make it into a variable. You'll see this used properly in the next section.

1.1.10. Sorting with a custom sorting order.

Say you with to sort a `vector` in descending order, or you have something more complicated in mind. Well the `sort()` function has an extra parameter to supply your own sorting order.

```C++
1   #include <bits/stdc++.h>
2   using namespace std;
3
4   int main(){
5
6     int arr[] = {3,4,6,2,5,1};
7     vector<int> v = {6,2,4,5,1,3};
8
9     sort(arr, arr+6, greater<int>());//Sorts the array {6,5,4,3,2,1}
10    sort(v.begin(), v.end(), greater<int>());//Sorts the vector {6,5,4,3,2,1}
11
12    return 0;
13  }
```

The `greater<int>()` function returns true when for the 2 inputs `a` and `b`, `a > b`. Using sort this way ensure that all elements make your comparator function `true`.

Let's say you want to sort a `vector<pair<int,int>>` such that the second element is sorted in ascending order and only if they are equal are the first elements sorted in descending order. Here's how you could go about it, this will also demonstrate how to use lambda expressions:

```C++
#include <bits/stdc++.h>
using namespace std;

int main(){

  int n;
  cin >> n;
  vector<pair<int,int>> v;
  for(int i = 0; i < n; i++)
    cin >> v[i];

  sort(v.begin(), v.end(), [](const pair<int,int> &a, const pair<int,int> &b)
  {
    return a.second < b.second || a.second == b.second && a.first > b.first)
  });

  return 0;
}
```

1.2. CSES Practice Questions

1.2.1. Distinct Numbers

Question

Question - Distinct Numbers Backup Link

Explanation

Accept all the numbers and insert them into a set. Then report the size of the set. This works due to the fact that a set only stores unique elements and removes duplicates.

More about sets can be found in the **Sets** section of Concepts.

Solution

```C++

#include <bits/stdc++.h>
using namespace std;

int main(){
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  cout.tie(0);

  int n;
```

```
11    cin >> n;
12    set<int> s;
13    for(int i = 0; i < n; i++){
14      int x;
15      cin >> x;
16      s.insert(x);
17    }
18    cout << s.size() << endl;
19    return 0;
20  }
```

1.2.2. Apartments

Question - Apartments Backup Link

Explanation :

The algorithm sorts both applicants and apartments, then uses a two pointer approach to match each applicant with the smallest available apartment whose size differs by at most `k`. If an apartment is too small, move to the next apartment; if it's too large, move to the next applicant. This greedy method ensures the maximum number of matches efficiently.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m, k;
    cin >> n >> m >> k;

    vector<int> applicants(n), apartments(m);

    // Read applicant preferences
    for (int i = 0; i < n; i++)
        cin >> applicants[i];

    // Read apartment sizes
    for (int i = 0; i < m; i++)
        cin >> apartments[i];

    // Sort both arrays
    sort(applicants.begin(), applicants.end());
    sort(apartments.begin(), apartments.end());

    int count = 0;
    int i = 0, j = 0;

    // Two-pointer approach to match applicants to apartments
    while (i < n && j < m) {
        // Check if current apartment fits current applicant's preference
        if (abs(applicants[i] - apartments[j]) <= k) {
            count++;
            i++;
            j++;
        }
```

```
        // If apartment is too small, try next apartment
        else if (applicants[i] - apartments[j] > k) {
            j++;
        }
        // If apartment is too big, try next applicant
        else {
            i++;
        }
    }

    cout << count << endl;
    return 0;
}
```

1.2.3. Ferris Wheel

Question - Ferris Wheel Backup Link

Explanation :

The algorithm sorts all weights, then uses two pointer, one at the lightest and one at the heaviest person, to form pairs without exceeding the limit. If they can share a gondola, both are removed; otherwise, the heavier one goes alone. This greedy pairing minimizes the total number of gondolas.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, x;
    cin >> n >> x;

    vector<int> weights(n);
    for (int i = 0; i < n; i++) {
        cin >> weights[i];
    }

    // Sort the weights
    sort(weights.begin(), weights.end());

    int gondolas = 0;
    int left = 0, right = n - 1;

    while (left <= right) {
        // If heaviest and lightest can share a gondola
        if (weights[left] + weights[right] <= x) {
            left++;
            right--;
        }
        // Otherwise, heaviest gets their own gondola
        else {
            right--;
        }
        gondolas++;
    }

    cout << gondolas << endl;
```

```
33
34      return 0;
35  }
36
```

1.2.4. Concert Tickets

Question - Concert Tickets Backup Link

Intuitive Explanation :

Store all ticket prices in a multiset to keep them sorted and allow duplicates. Each customer gives an offer, and you use `upper_bound` to find the first price strictly greater than that offer, then step one step back to get the best affordable ticket. If such a ticket exists, print it and remove it; otherwise print –1. This algorithm neatly handles each request without iterating through the whole list.

Code :

```C++
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, m, input;
    cin >> n >> m;

    // Multiset to store ticket prices in sorted order
    multiset<int> prices;

    // Store the m offers from customers
    vector<int> offers(m);

    // Insert the n ticket prices into the multiset
    for (int i = 0; i < n; i++) {
        cin >> input;
        prices.insert(input);
    }

    // Read all offers
    for (int i = 0; i < m; i++)
        cin >> offers[i];

    // For each offer, try to find the best possible ticket
    for (int i = 0; i < m; i++) {

        // Find the first price strictly greater than the offer
        auto it = prices.upper_bound(offers[i]);

        // If upper_bound points to begin(), no ticket <= offer exists
        if (it == prices.begin()) {
```

```
            cout << "-1" << endl;
        }
        else {
            // Move iterator to the largest price <= offer
            --it;

            // Output that price
            cout << *it << endl;

            // Remove that ticket so it can't be reused
            prices.erase(it);
        }
    }
}
```

1.2.5. Restaurant Customers

Question - Restaurant Customers Backup Link

Explanation :

The algorithm sorts all arrival and departure times, then uses two pointers to simulate guests entering and leaving. Each arrival increases the current count, and each departure decreases it. The maximum value reached during this sweep gives the peak number of guests present simultaneously.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;
    vector<int> arrivals(n), departures(n);
    for (int i = 0; i < n; ++i) {
        cin >> arrivals[i] >> departures[i];
    }

    // Sort arrival and departure times
    sort(arrivals.begin(), arrivals.end());
    sort(departures.begin(), departures.end());

    int i = 0, j = 0, curr = 0, ans = 0;
    // Sweep through both arrays to find maximum overlap
    while (i < n && j < n) {
        if (arrivals[i] < departures[j]) {
            curr++;      // new guest arrives
            ans = max(ans, curr);
            i++;
        } else {
            curr--;      // a guest departs
            j++;
        }
    }

    cout << ans << '\n'; // maximum guests present at once
```

```
33      return 0;
34  }
35
```

1.2.6. Movie Festival

Question - Movie Festival Backup Link

Intuitive Explanation :

We store each movie as a pair of (end_time, start_time) and sort by end_time so we can always consider the earliest finishing movie first. The greedy approach works because picking the movie that ends earliest leaves maximum time for future movies.

We iterate through all movies, watching one only if it starts after the previous one ends. Each time we do, we increment our count and update the latest end time, ensuring the optimal number of movies are chosen.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;

    // Store each movie as a pair (end_time, start_time)
    vector<pair<int, int>> movies(n);
    for (int i = 0; i < n; ++i) {
        int start, end;
        cin >> start >> end;
        movies[i] = {end, start};
    }

    // Sort movies by their ending time (greedy choice)
    sort(movies.begin(), movies.end());

    int maxMovies = 0;
    int currentEnd = 0;  // The end time of the last watched movie

    // Iterate through all movies
    for (auto [end, start] : movies) {
        // If the current movie starts after or exactly when the previous one ended
        if (start >= currentEnd) {
            maxMovies++;       // Watch this movie
            currentEnd = end;  // Update the end time to this movie's end
        }
    }

```

```
    cout << maxMovies << endl; // Output the result
    return 0;
}

```

1.2.7. Sum of Two Values

Question - Sum of Two Values Backup Link

Explanation :

The algorithm sorts all numbers, then uses two pointers—one starting at the smallest and one at the largest value—to find a pair that sums to the target. If the sum is too small, the left pointer moves right; if too large, the right pointer moves left. This efficiently finds the correct pair in linear time after sorting.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, target;
    cin >> n >> target;

    // Store each number along with its original index
    vector<pair<int, int>> nums(n);
    for (int i = 0; i < n; i++) {
        // number value
        cin >> nums[i].first;

        // original position (1-based index)
        nums[i].second = i + 1;
    }

    // Sort numbers by value to apply two-pointer technique
    sort(nums.begin(), nums.end());

    int left = 0, right = n - 1;
    while (left < right) {
        int sum = nums[left].first + nums[right].first;

        // If target sum found, print their original indices
        if (sum == target) {
            cout << nums[left].second << " " << nums[right].second;
            return 0;
        }
        // Move pointers based on comparison with target
        else if (sum < target) left++;
        else right--;
```

```
    }

    // If no valid pair found
    cout << "IMPOSSIBLE";
    return 0;
}

```

1.2.8. Maximum Subarray

Question - Maximum Subarray Sum Backup Link

Intuitive Explanation :

The algorithm finds the maximum possible sum of a continuous sequence in an array. It begins by assuming the first element is the best sum. Then, as it moves through the array, it decides whether to keep adding to the current streak or start fresh from the current number. At each step, it updates the overall best sum found so far, ensuring the final answer is the largest contiguous total.

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;

    vector<long long> arr(n);
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    // max_current = maximum subarray sum ending at the current index
    // max_global = overall maximum subarray sum found so far
    long long max_current = arr[0];
    long long max_global = arr[0];

    // Iterate through the array
    for (int i = 1; i < n; i++) {
        // Either start a new subarray at arr[i] or extend the existing one
        max_current = max(arr[i], max_current + arr[i]);

        // Update the global maximum if needed
        max_global = max(max_global, max_current);
    }

    // Output the maximum subarray sum
    cout << max_global << endl;
    return 0;
}

```

```
32
```

1.2.9. Stick Lengths

Question - Stick Lengths Backup Link

Intuitive Explanation :

The program minimizes the total adjustment cost to make all sticks equal in length. It sorts the stick lengths and picks the median as the target length since the median minimizes the sum of absolute differences. Unlike the mean, which minimizes squared differences, the median ensures minimal total movement for all sticks.

That might sound technical and complicated, so here's an easier way to picture it:

Intuitively, the median balances the values — half the sticks are shorter and half are longer — so adjusting everything toward it requires the least total effort. If you chose the mean, extreme stick lengths would pull the target toward them, increasing the total distance everyone else has to adjust, whereas the median stays steady and fair, unaffected by outliers.

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;

    vector<int> v(n);
    for (int i = 0; i < n; i++)
        cin >> v[i];

    // Sort the array in ascending order
    sort(v.begin(), v.end());

    // Choose the middle element (median) as the central value
    int c = v[v.size() / 2];

    long long ans = 0;
    // Calculate the total distance of all elements from the median
    for (int i = 0; i < n; i++)
        ans += abs(v[i] - c);

    // Output the minimum total distance
    cout << ans;
}
```

1.2.10. Missing Coin Sum

Question - Missing Coin Sum Backup Link

Intuitive Explanation :

Sorting the Coins: By sorting the coins in non-decreasing order, we can process them greedily.

Greedy Approach: Initialize a variable `sumSoFar` to 0, representing the maximum sum we can create with the coins processed so far.

For each coin value `currCoin` : If `currCoin` is greater than `sumSoFar + 1`, it means we cannot create the sum `sumSoFar + 1` (since all remaining coins are too large). Thus, `sumSoFar` + 1 is the answer. Otherwise, add `currCoin to sumSoFar`, as we can now create all sums up to `sumSoFar + currCoin` by including or excluding `currCoin` in subsets.

If we process all coins without finding a gap, the smallest sum we cannot create is current_max + 1.

Why This Works: If we can create all sums from 0 to `sumSoFar`, and the next coin `currCoin` is at most `sumSoFar + 1`, we can extend the range of creatable sums to `sumSoFar + currCoin`. A gap occurs when a coin is too large to fill the next sum (`sumSoFar + 1`), making that sum impossible to form.

Code :

```C++
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    vector<long long> coins(n);
    for (int i = 0; i < n; i++) cin >> coins[i];
    sort(coins.begin(), coins.end());

    long long sumSoFar = 0;
    // We can currently form all sums from 1 to sumSoFar.

    for (long long currCoin : coins) {
        // If the next needed sum is sumSoFar + 1 and currCoin is bigger,
        // we cannot fill that gap.
        if (currCoin > sumSoFar + 1) {
```

```
            cout << sumSoFar + 1 << "\n";
            return 0;
        }

        // Otherwise, currCoin helps us extend reachable sums.
        sumSoFar += currCoin;
    }

    // If all coins processed and no gap found, next unreachable sum is
    sumSoFar + 1.
    cout << sumSoFar + 1 << "\n";
    return 0;
}
```

1.2.11. Collecting Numbers

Question - Collecting Numbers Backup Link

Explanation :

The program stores the index of each number in the order it appears. It then scans numbers from 1 to n and checks whether a number appears before its predecessor. Whenever this happens, a new round is required. The final count represents the total number of rounds needed.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;

    vector<int> position(n + 1);
    for (int i = 0; i < n; i++) {
        int value;
        cin >> value;
        position[value] = i;
    }

    int rounds = 1;
    for (int i = 2; i <= n; i++) {
        if (position[i] < position[i - 1]) {
            rounds++;
        }
    }

    cout << rounds;
}
```

1.2.12. Collecting Numbers II

Question - Collecting Numbers II Backup Link

Explanation :

This problem works with a permutation of numbers from 1 to n and asks how many rounds are needed to collect the numbers in increasing order. A new round is required whenever the position of a number x appears after the position of x+1 in the array. Initially, we scan the array and count how many such “breaks” exist to compute the number of rounds.

For each query, two positions in the array are swapped. A full recount after every swap would be too slow, so the key idea is to only update the parts of the array that are affected. Swapping two values only changes the order relations involving those values and their immediate neighbors (x-1, x, x+1). Before performing the swap, we subtract any existing breaks caused by these values. After the swap, we recompute and add back the new breaks.

By maintaining an array that stores the current position of each value, each check can be done in constant time. This allows every query to be processed efficiently, keeping the total complexity fast even for large inputs.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int n, m;
vector<int> arr;   // Stores the permutation
vector<int> pos;   // pos[x] = index where value x is currently located

// Checks whether x and x+1 form a "break"
// A break means x appears after x+1 in the array
bool isBreak(int x) {
    if (x < 1 || x >= n) return false;   // Out of valid range
    return pos[x] > pos[x + 1];
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    cin >> n >> m;

    arr.resize(n);
    pos.resize(n + 2);

```

```
    // Read the permutation and record positions of each value
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
        pos[arr[i]] = i;
    }

    // Initially, at least one round is needed
    int rounds = 1;

    // Count how many times x comes after x+1
    // Each such case increases the number of rounds
    for (int x = 1; x < n; x++) {
        if (isBreak(x)) {
            rounds++;
        }
    }

    // Process each swap query
    while (m--) {
        int a, b;
        cin >> a >> b;
        a--;
        b--;   // Convert to 0-based indexing

        int u = arr[a];
        int v = arr[b];

        // Only these values can affect the number of breaks
        // because other relative orders remain unchanged
        set<int> affected = {
            u - 1, u, u + 1,
            v - 1, v, v + 1
        };

        // Remove old breaks before swapping
        for (int x : affected) {
            if (isBreak(x)) {
                rounds--;
            }
        }

        // Perform the swap in the array
        swap(arr[a], arr[b]);

        // Update positions of the swapped values
```

```
69          swap(pos[u], pos[v]);
70
71          // Add new breaks after swapping
72          for (int x : affected) {
73              if (isBreak(x)) {
74                  rounds++;
75              }
76          }
77
78          // Output the current number of rounds
79          cout << rounds << '\n';
80      }
81
82      return 0;
83  }
```

1.2.13. Playlist

Question - Playlist Backup Link

Explanation :

The trick is to slide a window across the array while keeping all its elements distinct. A set tracks which songs are currently inside the window: if the next song is already present, we shrink the window from the left until the duplicate disappears. Otherwise we extend the window to include it. As the window grows and shrinks, we keep updating the maximum length, which becomes the length of the longest playlist with all unique songs.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;

    // Read the array
    vector<int> v(n);
    for (int i = 0; i < n; i++) {
        cin >> v[i];
    }

    // Sliding window to find the longest subarray with all distinct elements.
    set<int> window;     // stores current distinct elements inside the window
    int left = 0;        // left pointer of the window
    int right = 0;       // right pointer of the window
    int bestLen = 0;     // maximum size found

    // Expand the window using 'right'
    while (right < n) {
        // If v[right] already exists, shrink the window from the left
        // until v[right] can be inserted without duplicates.
        if (window.count(v[right])) {
            window.erase(v[left]);
            left++;
        }
        else {
            // Element is unique in the window — include it
```

```
            window.insert(v[right]);

            // Update max length
            bestLen = max(bestLen, right - left + 1);

            // Move right pointer forward
            right++;
        }
    }

    cout << bestLen;
    return 0;
}
```

1.2.14. Towers

Question - Towers Backup Link

Explanation :

The idea is to maintain the top blocks of all towers in a multiset. For each new block, place it on the leftmost tower whose top is strictly greater; if no such tower exists, you start a new one. This greedy strategy works because always using the smallest possible valid tower keeps future placements flexible. The number of towers equals the size of the multiset.

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    multiset<int> tops; // stores the current top element of each tower

    for (int i = 0; i < n; i++) {
        int x;
        cin >> x;

        // Find first tower whose top > x (we can place x on that tower)
        auto it = tops.upper_bound(x);

        if (it != tops.end()) {
            // Reuse this tower: remove old top and replace with x
            tops.erase(it);
        }
        // Start a new tower or update reused one with top = x
        tops.insert(x);
    }

    // Number of towers equals the number of distinct tops
    cout << tops.size() << '\n';

    return 0;
```

```
33 }
```

1.2.15. Traffic Lights

Question - Traffic Lights Backup Link

Intuitive Explanation :

The program simulates cutting a stick of length `a` at `b` given positions. It uses a multiset ms to store all cut points and another multiset lens to track segment lengths. After each cut, it removes the old segment and adds two new ones. Finally, it prints the length of the largest segment remaining after each cut.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int a, b, x;
    cin >> a >> b;

    multiset<int> ms, lens; // ms stores all cut positions, lens stores all segment lengths
    ms.insert(a); // rightmost boundary
    ms.insert(0); // leftmost boundary
    lens.insert(a); // initially one segment of length 'a'

    for (int i = 0; i < b; i++) {
        cin >> x;

        // Insert the new cut position and find its neighbors
        auto mid = ms.insert(x);
        auto first = prev(mid);
        auto last = next(mid);

        // Remove the old segment and add the two new smaller segments
        lens.erase(lens.find(*last - *first));
        lens.insert(*last - *mid);
        lens.insert(*mid - *first);

        // Output the largest segment length after each cut
        cout << *lens.rbegin() << " ";
    }
}

```

1.2.16. Distinct Values Subarrays

Question - Distinct Values Subarrays Backup Link

Explanation :

This code uses a sliding window to count subarrays with all distinct elements. The right pointer expands the window, while a frequency map tracks duplicates. If a duplicate appears, the left pointer shrinks the window until all elements are unique again.At each position, the number of valid subarrays ending there is added to the answer.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;

    vector<int> a(n);
    for (int i = 0; i < n; i++) {
        cin >> a[i];
    }

    // Frequency map for elements in the current window
    map<int, int> freq;

    int left = 0;          // Left pointer of the sliding window
    long long answer = 0;  // Total number of valid subarrays

    for (int right = 0; right < n; right++) {
        freq[a[right]]++;  // Add current element to the window

        // Shrink window until all elements are distinct
        while (freq[a[right]] > 1) {
            freq[a[left]]--;
            left++;
        }

        // Number of distinct subarrays ending at 'right'
        answer += (right - left + 1);
    }

    cout << answer;
```

```
33 }
```

1.2.17. Distinct Values Subsequences

Question - Distinct Values Subsequences Backup Link

Explanation :

For each distinct value with `occ` occurrences, we have `(occc + 1)` choices: exclude it (0 copies) or choose 1 of the `occ` identical copies to include. Multiplying choices for all distinct numbers gives total possible combinations including the empty subsequence. Subtract 1 to remove the empty subsequence case, leaving the count of all distinct-value subsequences.

Example :

For the array [1, 3, 5, 2, 9, 3, 2]

The frequency table stores -

Key	Value
1	1
2	2
3	2
5	1
9	1

The number of distinct value subsequences

$$\begin{aligned} &= (1+1)\cdot(2+1)\cdot(2+1)\cdot(1+1)\cdot(1+1) \\ &= 2\cdot 3\cdot 3\cdot 2\cdot 2 \\ &= 72 \end{aligned}$$

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;
const int MOD = 1e9 + 7;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n;
    cin >> n;

    // Count frequency of each distinct value
```

```
    map<ll, ll> freq;
    for (int i = 0; i < n; i++) {
        int x;
        cin >> x;
        freq[x]++;
    }

    // For each distinct value, we can include 0, 1, 2, ..., or occ copies
    // This gives (occ + 1) choices per value; multiply all choices together
    ll ans = 1;
    for (auto [num, occ] : freq) {
        ans = (ans * (occ + 1)) % MOD;
    }

    // Subtract 1 to exclude the empty subsequence
    ans = (ans - 1 + MOD) % MOD;
    cout << ans << '\n';

    return 0;
}
```

1.2.18. Josephus Problem I

Question - Josephus Problem I Backup Link

Explanation :

We store all people in a linked list, for efficient deletions while moving forward. An iterator walks through the list, skipping one person each time. When the iterator reaches the end, it wraps back to the beginning. Each erased element is printed in order.

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;

    list<int> circle;
    for (int i = 1; i <= n; i++)
        circle.push_back(i);

    auto it = circle.begin();

    while (!circle.empty()) {
        // move to the next person (skip one)
        it++;
        if (it == circle.end())
            it = circle.begin();

        cout << *it << " ";

        // erase returns iterator to next element
        it = circle.erase(it);

        if (it == circle.end() && !circle.empty())
            it = circle.begin();
    }

    return 0;
}
```

1.2.19. Josephus Problem II

Question - Josephus Problem II Backup Link

Explanation :

Solution Overview:

- Uses a Fenwick Tree (Binary Indexed Tree) to efficiently track which positions are still active
- Each position is initially marked as "1" (active), and changed to "0" when removed

Key Operations:

1. update(idx, delta): Marks a position as active (+1) or removed (-1)
2. query(idx): Returns count of active elements from position 1 to idx
3. findKth(k): Uses binary search on the Fenwick Tree to find the k-th active element in O(log n) time

Algorithm Flow:

- Start with all n positions active
- In each iteration, move k steps forward in the circular list of remaining elements
- Use `findKth` to map from the circular index to the actual position
- Output that position and mark it as removed
- Repeat until all elements are removed

Complexity: O(n log n) - n removals, each taking O(log n) for finding and updating

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

class FenwickTree {
    vector<int> tree;
    int n;
public:
    FenwickTree(int n) : n(n), tree(n + 1, 0) {}

    // Add delta to position idx (used to mark elements as active/inactive)
    void update(int idx, int delta) {
        for (; idx <= n; idx += idx & -idx)
            tree[idx] += delta;
    }

    // Get count of active elements in range [1, idx]
    int query(int idx) {
        int sum = 0;
        for (; idx > 0; idx -= idx & -idx)
```

```
            sum += tree[idx];
        return sum;
    }

    // Binary search to find the k-th active element (1-indexed)
    int findKth(int k) {
        int pos = 0, bit = 1;
        // Find the highest power of 2 <= n
        while (bit <= n) bit <<= 1;
        bit >>= 1;

        // Binary search from highest bit to lowest
        for (; bit > 0; bit >>= 1) {
            if (pos + bit <= n && tree[pos + bit] < k) {
                k -= tree[pos + bit];
                pos += bit;
            }
        }
        return pos + 1;
    }
};

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    cin >> n >> k;

    // Initialize Fenwick Tree and mark all positions as active
    FenwickTree ft(n);
    for (int i = 1; i <= n; i++)
        ft.update(i, 1);

    int remaining = n;  // Track how many elements are still active
    int idx = 0;        // Current position in the circular arrangement

    while (remaining > 0) {
        // Move k steps forward in circular manner
        idx = (idx + k) % remaining;

        // Find the actual position of the (idx+1)-th active element
        int pos = ft.findKth(idx + 1);
        cout << pos << " ";

```

```
65          // Mark this position as removed
66          ft.update(pos, -1);
67          remaining--;
68      }
69
70      cout << "\n";
71      return 0;
72  }
```

1.2.20. Nested Ranges Check

Question - Nested Ranges Check Backup Link

Solution:

The algorithm uses a clever sorting trick: ranges are stored as ((l, -r), index) so that when sorted, ranges with the same left endpoint have larger right endpoints first. This ordering is crucial for the sweep line approach.

For detecting "contained" ranges, it sweeps left to right tracking the maximum right endpoint seen so far. If the current range's right endpoint is $\leq$ maxRight, it means this range is contained by a previous one (since they have l_current $\geq$ l_previous and r_current $\leq$ r_previous).

For detecting "contains" ranges, it sweeps right to left tracking the minimum right endpoint. If the current range's right endpoint is $\geq$ minRight, it contains at least one subsequent range (the range extends at least as far right as some range with a greater or equal left endpoint).

The time complexity is O(n log n) due to sorting, and space complexity is O(n). The key insight is that the special sorting allows both containment checks to be done in linear time after sorting.

C++

```cpp
#include <bits/stdc++.h>
using namespace std;
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<pair<pair<int,int>, int>> ranges(n);
    for (int i = 0; i < n; i++) {
        int l, r;
        cin >> l >> r;
        ranges[i] = {{l, -r}, i};
    }
    sort(ranges.begin(), ranges.end());
    vector<int> contains(n, 0), contained(n, 0);
    int maxRight = INT_MIN;
    for (auto &[p, idx] : ranges) {
        int r = -p.second;
        if (r <= maxRight)
            contained[idx] = 1;
        maxRight = max(maxRight, r);
    }
    int minRight = INT_MAX;
    for (int i = n - 1; i >= 0; i--) {
```

```
25          int r = -ranges[i].first.second;
26          int idx = ranges[i].second;
27          if (r >= minRight)
28              contains[idx] = 1;
29          minRight = min(minRight, r);
30      }
31      for (int x : contains) cout << x << " ";
32      cout << "\n";
33      for (int x : contained) cout << x << " ";
34      cout << "\n";
35  }
```

1.2.21. Nested Ranges Count

Question - Nested Ranges Count Backup Link

Hint:

Try sorting the intervals in ascending order of left index and descending order of right index. What algorithm could you come up with where you only have to iterate once through the list to get the answer? Think about why this sorting method was mentioned and what advantages it has.

Solution:

The problem asks us to find two values for each range: the number of other ranges it contains, and the number of other ranges that contain it. A naive solution comparing every pair would be too slow ($O(n^2)$). We'll need a better approach.

Say we were to sort every interval in ascending order of their left index and in descending order of their right index. This seems pretty random, however if you think about the first range in the sorted list, you can guarantee that any range that comes after it will be inside it simply by checking if it's right index is less than the right index of the first range.

This applies to all ranges in the sorted list i.e. for any range i, any subsequent range j will be contained by i if the right index of range j is less than i.

Likewise, the converse also applies, for any range i, the **count** of ranges that i is **contained** by is all ranges j, where $j < i$, such that right index of j is more than right index of i.

Let's look at an example to understand this better:

Say we're given the following ranges, for now they're going to be sorted in the order we want

$$\{\{1, 10\}, \{2, 8\}, \{2, 6\}, \{5, 7\}\}$$

Now the first range cannot be contained by anyone, because so the answer for range 0 is 0.

Range 1 is contained by Range 0 because $10 > 8$, so the answer for range 1 is 1.

Range 2 is contained by both range 0 and 1 because $10 > 6$ and $8 > 6$. Therefore the answer for range 2 is 2.

Lastly range 3 is contained by ranges 0 and 1 but **not** by range 2 because $10 > 7$, $8 > 7$, but $6 < 7$. Therefore the answer for range 3 is 2.

The output of the second line for the question hence would be $0, 1, 2, 2$

This however only ends up solving half the question, we still need to find out for every range i, how many ranges it contains.

For this we can use the same sorting order, just iterate in reverse. The reasoning is the same as before: for any range i, the **count** of ranges that i **contains** is the number of ranges j, where $j > i$ such that right index of j is less than the right index of i.

Using the ranges given previously, range 3 can contain no other ranges, therefore it's answer is 0.

Range 2 does **not** contain Range 3 because $7 > 6$. Therefore its answer is 0.

Range 1 contains both range 2 and range 3 because $6 < 8$ and $7 < 8$. Therefore its answer is 2.

Lastly range 0 contains range 1, range 2, and range 3, because $8 < 10$, $6 < 10$, and $7 < 10$. Therefore it's answer is 3.

While the approach seems logical, how can we efficiently find out how many ranges have a right end point less than the current range or greater than the current range. For that, we can use a Fenwick tree as an indexed set. You can add the right index of all ranges either succeeding or preceding and then find the how many right indexes are more or less than the current range.

Looking at the code should make it much clearer:

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> fen;

struct Range{
  int l, r, idx;

  bool operator<(const Range& ran){
    return l < ran.l || l == ran.l && r > ran.r;
  }
};

void add(int x, int k){
  for(; x < fen.size(); x += x & -x)
    fen[x] += k;
}

int sum(int x){
  int ans = 0;
  for(; x > 0; x -= x & -x)
    ans += fen[x];
  return ans;
}

int main(){
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  cout.tie(0);
  //freopen("NestedRangesCount.in","r",stdin);
  //freopen("NestedRangesCount.out","w",stdout);
  int n;
  cin >> n;
```

```
34
35  vector<Range> v(n);
36  vector<int> comp;
37
38  for(int i = 0; i < v.size(); i++){
39    cin >> v[i].l >> v[i].r;
40    v[i].idx = i;
41    comp.push_back(v[i].r);
42  }
43
44  //Index compression for the right endpoints:
45  sort(comp.begin(), comp.end());
46  comp.erase(unique(comp.begin(), comp.end()), comp.end());
47
48  for(int i = 0; i < v.size(); i++)
49    v[i].r = lower_bound(comp.begin(), comp.end(), v[i].r) - comp.begin() +
      1;
50
51  sort(v.begin(), v.end());
52  fen.resize(comp.size() + 1);
53
54  vector<int> c(n), d(n);// c[i] is the number of ranges v[i] contains, d[i]
    is the number of ranges that v[i] is contained by.
55
56  for(int i = v.size()-1; i >= 0; i--){
57    // for every range, add the number of ranges with r < v[i].r. l > v[i].r
      because of the sorting order.
58    // if l = v[i].l , then the smaller ranges will get added first to
      correctly find c[i] because of the sorting order.
59    // sum(v[i].r) - 1 gives that number.
60    add(v[i].r,1);
61    c[v[i].idx] = sum(v[i].r) - 1;
62  }
63
64  //Resetting the Fewnick tree.
65  fen.clear();
66  fen.resize(comp.size() + 1);
67
68  for(int i = 0; i < v.size(); i++){
69    // for every range, add the number of ranges with r > v[i].r. l < v[i].l
      because of the sorting order.
70    // if l = v[i].l , then the larger ranges will get added first to
      correctly find d[i] because of the sorting order.
71    // sum(fen.size() - 1) - sum(v[i].r-1) - 1 gives that number.
72    add(v[i].r,1);
73    d[v[i].idx] = sum(fen.size()-1) - sum(v[i].r-1) - 1;
```

```
74    }
75
76    for(int i = 0; i < c.size(); i++)
77      cout << c[i] << " ";
78    cout << endl;
79
80    for(int i = 0; i < d.size(); i++)
81      cout << d[i] << " ";
82    cout << endl;
83
84    return 0;
85  }
```

1.2.22. Room Allocation

Question - Room Allocation Backup Link

Explanation :

We use a greedy algorithm by sorting customers by their arrival time. For each customer, we check if any previously used room has become free (i.e., its last guest departed before the current guest arrives). We use a multiset to efficiently track rooms by their end times - if a suitable free room exists, we reuse it; otherwise, we allocate a new room. This greedy choice is optimal because assigning an available room to the earliest arriving customer never leads to a worse solution than leaving it empty.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;

    // Store each customer's booking: {start_time, end_time, original_index}
    vector<tuple<int, int, int>> bookings(n);

    for (int i = 0; i < n; i++) {
        int start, end;
        cin >> start >> end;
        bookings[i] = {start, end, i};
    }

    // Sort bookings by start time
    sort(bookings.begin(), bookings.end());

    // Track available rooms: {end_time, room_number}
    // Sorted by end_time to find rooms that become free earliest
    multiset<pair<int, int>> availableRooms;

    vector<int> assignedRoom(n);
    int totalRooms = 0;

    for (const auto& [start, end, originalIndex] : bookings) {
        int roomNumber;

        // Find a room that's free before this booking starts
```

```cpp
        // upper_bound finds first room with end_time > start
        // We want the room with end_time <= start, so we go one back
        auto it = availableRooms.upper_bound({start, INT_MAX});

        if (it == availableRooms.begin()) {
            // No available room found - need a new room
            roomNumber = ++totalRooms;
        } else {
            // Reuse an existing room that's now free
            --it;
            roomNumber = it->second;
            availableRooms.erase(it);
        }

        // Mark this room as occupied until 'end' time
        availableRooms.insert({end, roomNumber});
        assignedRoom[originalIndex] = roomNumber;
    }

    // Output results
    cout << totalRooms << "\n";
    for (int room : assignedRoom) {
        cout << room << " ";
    }
    cout << "\n";

    return 0;
}
```

1.2.23. Factory Machines

Question - Factory Machines Backup Link

Explanation :

The key idea is that the number of items produced increases monotonically with time, so we can binary-search the minimum time needed to make at least `t` items. For any guessed time `mid`, we compute how many items all machines together can produce by summing $\frac{\texttt{mid}}{v[i]}$. If the total is $\geq$ t, we try a smaller time; otherwise, we increase the time. This guarantees we find the earliest moment when production meets the target.

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {

    ll n, t;
    cin >> n >> t;

    // v[i] = time taken by machine i to produce ONE item
    vector<ll> v(n);
    for (int i = 0; i < n; i++) {
        cin >> v[i];
    }

    // Binary search on time.
    // low  = minimum possible time
    // high = a very large upper bound (1e18 works for all constraints)
    ll low = 1, high = 1e18, ans = -1;

    while (low <= high) {
        ll mid = (low + high) / 2;

        // Count how many items all machines can produce in 'mid' time
        ll total = 0;
        for (int i = 0; i < n; i++) {
            total += mid / v[i];

            // If already enough, stop early (avoid overflow + speedup)
            if (total >= t) break;
        }
```

```
32
33          // If we can produce at least t items in 'mid' time,
34          // try to find an even smaller valid time.
35          if (total >= t) {
36              ans = mid;
37              high = mid - 1;
38          }
39          else {
40              // Not enough items — need more time.
41              low = mid + 1;
42          }
43      }
44
45      cout << ans << "\n";
46      return 0;
47  }
```

1.2.24. Tasks and Deadlines

Question - Tasks and Deadlines Backup Link

Explanation :

In this problem, we must schedule tasks to maximize total reward, where each task gives a reward only if completed before its deadline.

The intuitive greedy approach is to always prioritize tasks with the shortest durations first, because choosing a long task early delays all subsequent tasks and reduces their chances of meeting deadlines. By sorting tasks by duration and maintaining a timeline, we ensure we fit the maximum number of tasks in the shortest possible time. Whenever adding a new task would exceed its deadline, we can replace the longest task in our schedule with it if it has a smaller duration. Thus, the algorithm minimizes wasted time and maximizes the number of completed tasks for optimal total reward.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    int n;
    cin >> n;

    // jobs[i] = {duration, deadline}
    vector<pair<int, int>> jobs(n);
    for (int i = 0; i < n; i++) {
        cin >> jobs[i].first >> jobs[i].second;
    }

    // Sort by duration (or by first element), ensures earliest finishing attempts first
    sort(jobs.begin(), jobs.end());

    ll time_elapsed = 0;   // running sum of durations
    ll total_reward = 0;   // accumulated reward

    for (int i = 0; i < n; i++) {
        time_elapsed += jobs[i].first;              // finish this job at this time
        total_reward += jobs[i].second - time_elapsed;  // reward = deadline - completion time
    }
```

```
25
26      cout << total_reward;
27  }
```

1.2.25. Reading Books

Question - Reading Books Backup Link

Explanation :

If one book takes more than half the total time, one child will be forced to wait while the other finishes that long book. Otherwise, they can optimally interleave their reading with no idle time. Thus, the answer is $\max(\text{total\_time}, 2 * \text{longest\_book})$.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;

    vector<long long> books(n);
    long long total = 0, max_book = 0;

    // Read book times, track:
    // 1) total time of all books
    // 2) the longest single book
    for (int i = 0; i < n; i++) {
        cin >> books[i];
        total += books[i];
        max_book = max(max_book, books[i]);
    }

    // Minimum total time is governed by:
    // - total (one person reading sequentially)
    // - OR twice the largest book (two-person parallel reading constraint)
    // The answer is the max of these two.
    cout << max(total, 2 * max_book) << "\n";

    return 0;
}
```

1.2.26. Sum of Three Values

Question - Sum of Three Values Backup Link

Explanation :

This solution finds three numbers that sum to the target by first sorting the array and then fixing one element at a time. For each fixed element, it uses a two-pointer scan on the remaining range to efficiently search for a complementary pair. Sorting allows the sum to guide pointer movement, reducing the search from cubic to quadratic time. If no valid triple exists, the answer is declared impossible, keeping the logic clean and deterministic.

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, target;
    cin >> n >> target;

    // Store {value, original_index}
    vector<pair<int, int>> v(n);

    for (int i = 0; i < n; i++) {
        cin >> v[i].first;
        v[i].second = i + 1;  // keep original 1-based index
    }

    // Sort by value so we can use the two-pointer trick
    sort(v.begin(), v.end());

    // Fix one element v[i], then find two others using two pointers
    for (int i = 0; i < n - 2; i++) {
        int l = i + 1;       // left pointer
        int r = n - 1;       // right pointer

        while (l < r) {
            int sum = v[i].first + v[l].first + v[r].first;

            if (sum == target) {
                // Output original positions (not sorted ones)
                cout << v[i].second << " " << v[l].second << " " <<
                v[r].second;
                return 0;
```

```
            }
            else if (sum < target) {
                l++;         // need a larger sum → move left pointer right
            }
            else {
                r--;         // need a smaller sum → move right pointer left
            }
        }
    }

    // If no triple found
    cout << "IMPOSSIBLE";
    return 0;
}
```

1.2.27. Sum of Four Values

Question - Sum of Four Values Backup Link

Explanation :

If you understood the above three sum algorithm, four sum simply extends this by fixing two elements instead of one. You iterate through all pairs (i, j), then for each pair, use the same two-pointer technique to find the remaining two numbers that complete the target sum.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, target;
    cin >> n >> target;

    // Store (value, original_index)
    vector<pair<int, int>> nums(n);

    for (int i = 0; i < n; i++) {
        cin >> nums[i].first;
        nums[i].second = i + 1;   // 1-based indexing as required by CSES
    }

    // Sort by value to enable two-pointer scanning later
    sort(nums.begin(), nums.end());

    // Fix first two values using indices i and j
    for (int i = 0; i < n - 3; i++) {
        for (int j = i + 1; j < n - 2; j++) {

            int left = j + 1;
            int right = n - 1;

            // Two-pointer search for remaining pair
            while (left < right) {
                long long sum = nums[i].first
                + nums[j].first
                + nums[left].first
                + nums[right].first;

                if (sum == target) {
```

```
                    // Output original positions
                    cout << nums[i].second << " "
                    << nums[j].second << " "
                    << nums[left].second << " "
                    << nums[right].second;
                    return 0;
                }

                // Adjust pointers based on sum size
                if (sum < target) {
                    left++;
                } else {
                    right--;
                }
            }
        }
    }

    cout << "IMPOSSIBLE";
    return 0;
}
```

1.2.28. Nearest Smaller Values

Question - Nearest Smaller Values Backup Link

Intuitive Explanation :

We use a set of pairs (value, index) to maintain a sorted collection of elements seen so far. The lower_bound function efficiently locates the first element not smaller than the current value, allowing quick access to the previous smaller element by moving one step back. After each iteration, larger or equal elements are erased to maintain order and correctness.

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ll n, a;
    cin >> n;
    set<pair<ll,ll>> s;  // Stores pairs of (value, index) in sorted order by value

    for (int i = 0; i < n; i++) {
        cin >> a;

        // Find the first element in the set whose value >= current value 'a'
        auto it = s.lower_bound({a, -1});

        // If there's no smaller value, output 0
        if (it == s.begin()) cout << "0 ";
        else {
            // Otherwise, go one step back to get the last smaller element
            --it;
            cout << it->second + 1 << " ";  // Output its index (1-based)
        }

        // Remove all elements with value >= 'a' (not needed anymore)
        auto it2 = s.lower_bound({a, -1});
        s.erase(it2, s.end());

        // Insert current element (value, index)
        s.insert({a, i});
    }
}
```

```
33
```

1.2.29. Subarray Sums I

Question - Subarray Sums I Backup Link

Explanation :

All numbers are positive, so we keep a sliding window: expand the right end, and whenever the sum exceeds x we shrink from the left until it fits. Each time the window sum equals x we have one valid subarray.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ll n, x;
    cin >> n >> x;

    vector<int> v(n);
    for (int i = 0; i < n; i++) {
        cin >> v[i];
    }

    ll curr = 0;   // current window sum
    ll count = 0;  // number of subarrays equal to x
    ll l = 0;      // left pointer of sliding window

    for (int r = 0; r < n; r++) {
        curr += v[r];               // expand window to the right

        // shrink window from the left while sum exceeds x
        while (curr > x) {
            curr -= v[l];
            l++;
        }

        // if current window sum matches x, count it
        if (curr == x) count++;
    }

    cout << count;
}
```

1.2.30. Subarray Sums II

Question - Subarray Sums II Backup Link

Explanation :

We iterate through the array while maintaining a running prefix sum. A map stores how many times each prefix sum has appeared so far. At each position, if a previous prefix sum equals `currentSum` − `targetSum`, a subarray with the required sum exists. We add its frequency to the answer and then record the current prefix sum. This counts all valid subarrays in linear time.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

using ll = long long;

int main() {
    ll n, targetSum;
    cin >> n >> targetSum;

    vector<ll> array(n);
    for (int i = 0; i < n; i++) {
        cin >> array[i];
    }

    // prefixSumCount[s] = how many times prefix sum 's' has occurred
    map<ll, ll> prefixSumCount;
    prefixSumCount[0] = 1;   // empty prefix

    ll currentSum = 0;
    ll subarrayCount = 0;

    for (int i = 0; i < n; i++) {
        currentSum += array[i];

        // count subarrays ending here with sum = targetSum
        subarrayCount += prefixSumCount[currentSum - targetSum];

        // record current prefix sum
        prefixSumCount[currentSum]++;
    }

```

```
    cout << subarrayCount << '\n';
    return 0;
}
```

1.2.31. Subarray Divisibility

Question - Subarray Divisibility Backup Link

Explanation :

We use prefix sums modulo `n` to count subarrays whose sum is divisible by `n`. Each element is first reduced modulo `n` to keep values small. As we iterate, we maintain the current prefix sum modulo `n`. A map stores how many times each modulo value has appeared so far. If the same modulo appears again, the subarray between them has sum divisible by `n`. We add the frequency of the current modulo to the answer and update the map.

Code :

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long n;
    cin >> n;

    vector<long long> arr(n);
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
        arr[i] %= n;                // keep values within modulo n
    }

    unordered_map<long long, long long> frequency;
    frequency[0] = 1;               // empty prefix sum

    long long prefixSum = 0;
    long long result = 0;

    for (int i = 0; i < n; i++) {
        prefixSum = (prefixSum + arr[i]) % n;
        if (prefixSum < 0) prefixSum += n;

        result += frequency[prefixSum];
        frequency[prefixSum]++;
    }

    cout << result;
    return 0;
}
```

1.2.32. Distinct Values Subarrays II

Question - Subarray Distinct Values Backup Link

Explanation :

Use a sliding window with a frequency map. Expand the right end; if the number of distinct elements exceeds k, shrink from the left until it doesn't. Every position contributes `window_length` new subarrays ending there, so we add that to the answer.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    long long n, k;
    cin >> n >> k;

    vector<long long> v(n);
    for (int i = 0; i < n; i++) {
        cin >> v[i];
    }

    map<long long, int> freq;          // frequency of each element in current window
    long long left = 0, ans = 0, distinct = 0;

    for (int right = 0; right < n; right++) {
        // add right element to window
        if (++freq[v[right]] == 1) {
            distinct++;
        }

        // shrink window until we have at most k distinct elements
        while (distinct > k) {
            freq[v[left]]--;
            if (freq[v[left]] == 0) {
                distinct--;
            }
            left++;
        }

        // all subarrays ending at right with start in [left, right] are valid
```

```
32          ans += (right - left + 1);
33      }
34
35      cout << ans << "\n";
36  }
```

1.2.33. Array Division

Question - Array Division Backup Link

Explanation :

This solution minimizes the largest subarray sum when dividing the array into `k` consecutive subarrays using binary search. It establishes bounds where the answer lies between the largest element and the total sum of the array. For each candidate sum (mid), it checks if it's possible to split the array into at most `k` subarrays without any subarray exceeding that sum. If possible, a lower sum is attempted; otherwise, a higher sum is tested. Finally, the smallest feasible maximum subarray sum is output as the answer.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    cin >> n >> k;
    vector<int> a(n);

    ll left = 0, right = 0;
    // 'left' = minimum possible max sum (largest single element)
    // 'right' = maximum possible max sum (sum of all elements)

    for (int i = 0; i < n; i++) {
        cin >> a[i];
        left = max(left, (ll)a[i]);
        right += a[i];
    }

    ll ans = right;   // Best answer found via binary search

    while (left <= right) {
        ll mid = (left + right) / 2;

        // Try to split into <= k subarrays where no subarray sum exceeds 'mid'
        int subarrays = 1;
```

```
        ll current_sum = 0;
        bool possible = true;

        for (int i = 0; i < n; i++) {
            // If adding this element exceeds 'mid', start a new subarray
            if (current_sum + a[i] > mid) {
                subarrays++;
                current_sum = a[i];

                // Too many subarrays ⇒ mid is too small
                if (subarrays > k) {
                    possible = false;
                    break;
                }
            } else {
                current_sum += a[i];
            }
        }

        if (possible) {
            ans = mid;        // mid works ⇒ try to lower the maximum sum
            right = mid - 1;
        } else {
            left = mid + 1;   // mid too small ⇒ increase
        }
    }

    cout << ans << endl;
    return 0;
}
```

1.2.34. Sliding Median

Question - Sliding Median Backup Link

Explanation :

Maintain two multisets: `low` holds the smaller half (including the median) and `high` holds the larger half. After each insert/remove we rebalance so that `low` is never smaller than `high` and differs by at most one element. The median is always the largest element of `low`.

Code :

```C++
#include <bits/stdc++.h>
using namespace std;

multiset<int> lowSet, highSet;

void rebalance() {
    if (lowSet.size() > highSet.size() + 1) {
        highSet.insert(*lowSet.rbegin());
        auto it = lowSet.end();
        --it;
        lowSet.erase(it);
    } else if (lowSet.size() < highSet.size()) {
        lowSet.insert(*highSet.begin());
        highSet.erase(highSet.begin());
    }
}

void add(int x) {
    if (lowSet.empty() || x <= *lowSet.rbegin()) lowSet.insert(x);
    else highSet.insert(x);
    rebalance();
}

void remove_one(int x) {
    auto it = lowSet.find(x);
    if (it != lowSet.end()) lowSet.erase(it);
    else {
        it = highSet.find(x);
        highSet.erase(it);
    }
    rebalance();
}

```

```cpp
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int n, k;
    cin >> n >> k;
    vector<int> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];

    for (int i = 0; i < k; i++) add(a[i]);
    cout << *lowSet.rbegin();

    for (int i = k; i < n; i++) {
        add(a[i]);
        remove_one(a[i - k]);
        cout << " " << *lowSet.rbegin();
    }
    cout << "\n";
    return 0;
}
```

1.2.35. Sliding Cost

Question - Sliding Cost Backup Link

Explanation :

As in Sliding Median, keep two multisets split around the median but also track their sums. The total cost is `median * |low| - sumLow + sumHigh - median * |high|`. After each insertion/removal we rebalance and recompute using the maintained sums in O(1).

Code :

```C++
#include <bits/stdc++.h>
using namespace std;
using ll = long long;

multiset<int> lowSet, highSet;
ll sumLow = 0, sumHigh = 0;

void rebalance() {
    if (lowSet.size() > highSet.size() + 1) {
        int x = *lowSet.rbegin();
        sumLow -= x;
        lowSet.erase(prev(lowSet.end()));
        highSet.insert(x);
        sumHigh += x;
    } else if (lowSet.size() < highSet.size()) {
        int x = *highSet.begin();
        sumHigh -= x;
        highSet.erase(highSet.begin());
        lowSet.insert(x);
        sumLow += x;
    }
}

void add(int x) {
    if (lowSet.empty() || x <= *lowSet.rbegin()) {
        lowSet.insert(x);
        sumLow += x;
    } else {
        highSet.insert(x);
        sumHigh += x;
    }
    rebalance();
}
```

```
34
35 void remove_one(int x) {
36     auto it = lowSet.find(x);
37     if (it != lowSet.end()) {
38         sumLow -= x;
39         lowSet.erase(it);
40     } else {
41         it = highSet.find(x);
42         sumHigh -= x;
43         highSet.erase(it);
44     }
45     rebalance();
46 }
47
48 ll cost() {
49     int med = *lowSet.rbegin();
50     return med * 1LL * lowSet.size() - sumLow + sumHigh - med * 1LL *
       highSet.size();
51 }
52
53 int main() {
54     ios::sync_with_stdio(false);
55     cin.tie(nullptr);
56
57     int n, k;
58     cin >> n >> k;
59     vector<int> a(n);
60     for (int i = 0; i < n; i++) cin >> a[i];
61
62     for (int i = 0; i < k; i++) add(a[i]);
63     cout << cost();
64     for (int i = k; i < n; i++) {
65         add(a[i]);
66         remove_one(a[i - k]);
67         cout << " " << cost();
68     }
69     cout << "\n";
70     return 0;
71 }
```

1.2.36. Movie Festival II

Question - Movie Festival II Backup Link

Explanation :

The movies are sorted by ending time so earlier-finishing movies are considered first. A multiset stores when each of the k watchers becomes free. For each movie, we find the latest watcher free at or before its start time. If such a watcher exists, we assign the movie and update their free time. This greedy process maximizes the total number of movies watched.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, k;
    cin >> n >> k;

    // movies[i] = {end_time, start_time}
    vector<pair<int, int>> movies(n);
    for (int i = 0; i < n; i++) {
        cin >> movies[i].second >> movies[i].first;
    }

    // Sort movies by ending time (classic interval scheduling)
    sort(movies.begin(), movies.end());

    // Each element represents when a watcher becomes free
    multiset<int> freeAt;
    for (int i = 0; i < k; i++) {
        freeAt.insert(0);
    }

    int watched = 0;

    for (auto [endTime, startTime] : movies) {
        // Find a watcher who is free at or before startTime
        auto it = freeAt.upper_bound(startTime);

        if (it == freeAt.begin()) {
            // No watcher available
            continue;
        }
```

```
33
34          // Assign this movie to the latest possible free watcher
35          freeAt.erase(--it);
36          freeAt.insert(endTime);
37          watched++;
38      }
39
40      cout << watched;
41      return 0;
42  }
```

1.2.37. Maximum Subarray Sum II

Question - Maximum Subarray Sum II Backup Link

Explanation :

The code finds the maximum subarray sum whose length lies between `a` and `b`. It first builds a prefix sum array so any subarray sum can be computed in O(1). A multiset stores candidate prefix sums that can serve as valid subarray starts. As the right end moves forward, new valid prefixes are added to the set. Prefixes that would make the subarray longer than b are removed. The smallest prefix in the set gives the maximum possible subarray ending at the current index. The answer is updated by comparing all such valid subarrays efficiently.

Code :

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, minLen, maxLen;
    cin >> n >> minLen >> maxLen;

    vector<long long> values(n);
    for (int i = 0; i < n; i++) {
        cin >> values[i];
    }

    // Prefix sums: prefix[i] = sum of first i elements
    vector<long long> prefix(n + 1, 0);
    for (int i = 0; i < n; i++) {
        prefix[i + 1] = prefix[i] + values[i];
    }

    multiset<long long> candidates;
    long long bestSum = LLONG_MIN;

    for (int right = minLen; right <= n; right++) {
        // Add prefix corresponding to subarrays of length >= minLen
        candidates.insert(prefix[right - minLen]);

        // Remove prefix that would make subarray length > maxLen
        if (right > maxLen) {
            candidates.erase(candidates.find(prefix[right - maxLen - 1]));
        }
```

```
30
31          // Best subarray ending at 'right'
32          bestSum = max(bestSum, prefix[right] - *candidates.begin());
33      }
34
35      cout << bestSum << '\n';
36      return 0;
37  }
```